

Keeping Product Intent Connected in Agentic Software Development

The opportunity and approach behind Coral Reef Ventures

Whitepaper draft | October 2026

Executive overview

Software teams need to know what their product is supposed to do, whether a change preserves that promise, and whether users achieve the intended result. As AI agents take on more implementation work, those questions become harder to answer from code review and scattered documents alone.

Coral Reef Ventures is building open foundations and focused products that keep this understanding connected. Markset makes documents generated by agents readable and portable. Intentset records product behavior, implementation ownership and verification in the repository. Streamlane is being developed to coordinate human and agent work. Driftline is planned to connect releases with user experience and feedback. [1–3]

Our thesis is that the next opportunity in software development lies in maintaining a trustworthy connection between intent, execution and outcomes. A team should be able to ask what a change means for its product, see the evidence behind the answer, and decide what to do next. Agents can help maintain the records; people retain authority over product promises and consequential decisions.

Each offering can be adopted independently. Our longer-term vision connects them through explicit product meaning and evidence, allowing information created during planning to remain useful during implementation, release and learning.

This paper explains the problem, the opportunity, and how the approach could work in practice. It separates current foundations from proposed integrations and future capabilities. It does not claim that a connected product model guarantees correctness or that faster code generation always produces better outcomes.

Who this is for

Engineering and product leaders introducing agents into real delivery workflows, developers evaluating open foundations, and partners exploring the infrastructure needed for agentic software development.

The problem created by agentic development

Coding agents can make changes across files, run tools and prepare work for review. GitHub documents these capabilities alongside a continuing requirement for human review and validation. Anthropic similarly describes agents operating through tool use and feedback, with human checkpoints where needed. [4–5] The amount of implementation delegated will vary by team, codebase and task.

For teams delegating substantial work, the coordination problem extends beyond the coding agent. Product understanding often lives across tickets, specifications, code, tests, support articles and people. A change in one place does not automatically make the others current.

Product promises lose their connection to implementation

A ticket explains why a feature was requested. Code expresses what it now does. Tests verify selected cases. Customer guidance describes what users should expect. When these disagree, reviewers must reconstruct the relationship before they can judge the change. An agent working with incomplete context can preserve the local code pattern while missing the product rule behind it.

Review volume can exceed review capacity

When generating a change becomes easier, the team still has to understand its effects. A large diff can hide a small but consequential behavior change. A short AI summary can hide uncertainty just as easily. Review needs a stable account of the product, linked evidence and clear ownership, together with inspection of the implementation where risk warrants it.

Passing checks leave unanswered questions

A successful test run shows that the checks executed passed. It does not establish that every relevant rule has a check, that the checks cover the intended failure cases, or that users can accomplish the goal. Evidence becomes more useful when it identifies the behavior being checked and the exact code snapshot to which it applies.

Release feedback often arrives without product context

Errors, adoption metrics and user comments can tell a team that something is wrong. They may not explain which product promise is affected, who owns it, or which change needs attention. Without that connection, production learning becomes another queue of information to interpret.

These problems existed before coding agents. Our thesis is that greater delegation makes them harder to ignore: teams need a durable way to carry product understanding across the lifecycle, even when the people approving a change did not write its implementation.

The opportunity and the product approach

The opportunity is to reduce the effort required to establish what a product promises, assess a proposed change, and learn from its use. The first audience we would pursue is software teams already relying on agents enough to experience gaps in review, ownership or documentation. The immediate value should be visible on a single capability before broader adoption is necessary.

We organize the approach around four needs. Their relationship is a product vision; the complete connected workflow is not available today.

Need	Offering	Role
Readable documents	Markset	A small layout vocabulary over Markdown, with portable source and defined fallbacks.
Durable product meaning	Intentset	Repository records connecting behavior, rules, owning code and verification evidence.
Coordinated execution	Streamlane	Work management being developed for teams and agents, with GitHub and Slack integration.
Learning from use	Driftline	Planned release context, user feedback, error ownership, feature flags and betas.

Intentset supplies product meaning

Intentset is the proposed connection between these needs. A product behavior has an identity, rules, an accountable implementation slice and associated checks. Its draft export contract defines what other tools can read from the model. That creates a basis for integration without requiring a team to adopt every Coral Reef product. [2]

Markset makes that meaning easier to share

Markset adds a closed vocabulary of layout constructs to Markdown. Its tooling can check the structure of a document, render it, and produce a plain Markdown fallback. It can support readable plans, reports and explanations while keeping source reviewable. Layout validation does not validate the factual accuracy of the content. [3]

The connected workflow preserves context

Our vision carries approved intent into work, links implementation and checks to that intent, and relates release feedback to the same behavior. Markset can provide readable artifacts throughout. With each connection, teams spend less time rebuilding context from separate systems. This advantage depends on reliable identities, current records and integrations that preserve their meaning.

How Intentset supports informed review

Intentset starts with observable behavior expressed in product language. A promise should describe what a user can do and the rules that govern it. The implementation slice identifies the code and contracts responsible for delivering that behavior. Verification records name the checks and associate their results with a particular snapshot. [2]

People approve product meaning

Agents can draft a model from existing requirements, code and tests, then propose updates as they work. That initial reconstruction describes what the agent found; it does not prove that existing behavior is desirable. A person reviews the promises and rules and resolves ambiguities before treating them as approved intent. Intentset explicitly reserves approval for people. [2]

When an agent changes the implementation, the relevant records and checks should change in the same commit. The reviewer can examine changes in behavior, affected ownership, missing evidence and downstream explanations. The product model provides an additional basis for review; security-sensitive and complex implementation changes still require appropriate engineering scrutiny.

Checks expose gaps and stale evidence

Intentset's current specifications and toolchain include model validation, architecture checks, evidence and change impact. Drift checking can flag code changes without corresponding record updates. Evidence tied to a commit helps distinguish a current passing result from an older result or a check that has not run. [2]

These controls can establish structural consistency and expose omissions. They cannot establish completeness of the model or quality of every assertion. An agent could make code, records and tests agree around the same mistaken interpretation. Independent review of acceptance criteria and appropriate testing remain necessary. Anthropic's evaluation guidance also describes the value of combining automated, model-based and human judgment. [6]

Architecture and knowledge stay accountable

Intentset assigns behavior to owning slices and checks declared architectural boundaries. Its baseline approach allows existing violations to be recorded while preventing the baseline from growing. This supports incremental adoption in an existing codebase. The boundaries and baseline themselves need engineering review. [2]

The model also links customer explanations to their sources. When a source changes, related guidance can be held for review. This helps teams avoid publishing an explanation that still describes the previous behavior. Verification of an implementation and approval of a customer explanation remain separate decisions.

The intended result is a reviewer who can identify what changed, which evidence is relevant and what remains unknown, with direct access to the underlying records and implementation.

A change followed through the lifecycle

Consider a team adding scheduled exports to a business application. The user wants an export to run at a future time and remain subject to the organization's access rules. This is an illustrative scenario, not an implemented Coral Reef workflow or a customer result.

Agree the behavior and constraints

A product owner and agent draft the behavior in Intentset. An authorized user can schedule an export. Rules define time handling, cancellation, and what happens if access is revoked before execution. The product owner approves the intended behavior. Open questions stay visible rather than being silently settled by the implementation.

Coordinate and implement the work

In the proposed Streamlane integration, the task would reference the approved behavior and identify the responsible team. The implementing agent would load the relevant model, make changes in the owning slice, and update its records and checks. If the work requires changing a product rule, that proposed change would return for human approval.

Review the meaning and the evidence

Suppose the checks cover successful scheduling and cancellation, but no test verifies revoked access at execution time. A model that links each rule to its checks makes that omission easier to identify. The reviewer can request the missing evidence and inspect the authorization implementation before approving the change.

A readable change report could use Markset to present the changed behavior, affected rules, checks and unresolved questions. The report should link to its sources and distinguish failed checks, checks that have not run, and rules with no associated checks. A confident narrative alone would not justify approval.

Observe use and investigate friction

In the planned Driftline experience, the release would carry context about the scheduled-export behavior. The team could relate failures and adoption friction to that behavior and its owner. Feedback might show that users cannot tell when the export will run, even though the implementation passes its checks.

That observation would be a hypothesis to investigate. A metric by itself cannot establish the cause of friction. The team could examine errors and ask affected users, with appropriate consent and data controls, before changing the product.

Use learning to revise intent

If the problem is confirmed, a person could approve a clearer time display and confirmation message. That decision would update the behavior or guidance, create new work, and lead to fresh evidence for the next release. The connection to the original intent would remain available throughout.

This is the longer-term opportunity: agents work with explicit goals and constraints, while people can inspect the reasoning, evidence and user feedback behind the next decision.

Adoption and evidence of value

A useful starting point is one existing capability with meaningful rules and a recognizable owner. Let an agent reconstruct its behavior from the repository, review the model, and connect the checks that already exist. The initial question is whether this reveals something actionable and makes the next change easier to assess.

What exists today

Offering	Public status as of October 5 2026
Markset	Open source v0; the site lists reference version 0.4.2 and tooling for authoring, checking and rendering.
Intentset	Specifications in review; the site lists reference toolchain 0.6.1 for trying the model on a real capability.
Streamlane	In development; the parent site states that it is open to invited guests.
Driftline	In planning; the parent site states that it is open to invited guests.

These are published project statuses, not evidence of broad customer adoption or a completed integrated platform. Versions and availability will change. [1–3]

What the first pilot shows

Intentset reports a pilot on one Streamlane capability. The model validated and the architecture check passed. It identified two of three rules without tests and three behaviors checked only through the browser. This is a useful example of making verification gaps visible. It is a single reported internal pilot, with no demonstrated reduction in defects or review time. [2]

What to measure next

A pilot should compare the effort needed to assess comparable changes before and after adopting the model. Record time spent reconstructing context, important coverage gaps found, incorrect or stale records, and maintenance effort. Ask reviewers whether they can identify the affected promise, owner and current evidence without searching across several systems.

Track these measures alongside escaped defects and rework so that faster review does not conceal weaker review. If release feedback is later connected, evaluate whether teams can diagnose user friction and confirm improvement more reliably. Do not infer product success from adoption counts or test coverage alone.

How the approach can expand

Once a capability demonstrates value, extend the model to adjacent slices and automate record maintenance. Connect work management and reviewed explanations where those links reduce real effort. Integrate production feedback only when behavior identities, release mapping and data handling are reliable. Each step should justify its maintenance cost before the team adds more scope.

The vision and the conditions for success

Our vision is a software team that retains a shared understanding of its product while delegating more execution to agents. Approved promises provide context for work. Implementation changes carry current evidence. Releases remain connected to intended behavior. User experience informs the next product decision.

The commercial opportunity is to serve teams that find this connection valuable enough to adopt and sustain. Open foundations can lower the effort required to try the approach. Focused products can address coordination and learning needs as teams grow. The size of that opportunity and the willingness to pay remain questions to validate through real use.

The approach succeeds only if records remain readable and current, checks expose uncertainty, and integrations preserve ownership and evidence. Production feedback also requires explicit decisions about consent, access, retention and the data sent to agents. Humans need practical authority to approve, reject and revise the work.

We invite software teams to start with one capability: model its promises, connect the evidence, and assess the next change. We also welcome collaborators and partners who want to help establish open foundations for this way of building software.

Explore the projects at coralreefventures.com or contact hello@coralreefventures.com.

Sources and scope

Project descriptions and statuses were reviewed on October 5, 2026. The connected lifecycle, integration scenario and commercial opportunity are Coral Reef's proposed direction. External sources establish agent capabilities and the need for evaluation; they do not validate Coral Reef's approach.

[1] Coral Reef Ventures

<https://coralreefventures.com/>

[2] Intentset project overview and reported pilot

<https://intentset.org/>

[3] Markset project overview and tooling

<https://markset.org/>

[4] GitHub Application card for Copilot Agents

<https://docs.github.com/en/copilot/responsible-use/agents>

[5] Anthropic Building effective agents

<https://www.anthropic.com/engineering/building-effective-agents>

[6] Anthropic Demystifying evals for AI agents

<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>